



第三章 程序的转换及机器级表示

主要内容



华中科技大学

- ◆ 3.1 程序转换概述
- ◆ 3.2 IA-32指令系统概述
- ◆ 3.3 IA-32常用指令类型及其操作
- ◆ 3.4 C语言的机器级表示
- ◆ 3.5 复杂数据类型的分配和访问
- ◆ 3.6 越界访问和缓冲区攻击





3.3. IA-32常用指令类型及其操作

AT&T格式汇编指令的简单形式和后缀形式

简单形式	后缀形式	说明
mov (lea、add、inc、sub、dec等类似)	movb	有寄存器时，使用简单形式。 无寄存器时，使用后缀形式。 (若无寄存器时使用简单形式，会添加默认后缀l)
	movw	
	movl	
movsx	movsbw	使用后缀形式
	movsbl	
	movswl	
movzx	movzbw	使用后缀形式
	movzbl	
	movzwl	



华中科技大学

一、传送指令

功能：将数据、地址、立即数送入寄存器或存储器。

这类指令有：MOV、XCHG、LEA等。





1. 传送指令

格式: MOV OPS, OPD

功能: (OPS) $\rightarrow$ OPD。

注意:

寄存器 $\longleftrightarrow$ 寄存器; 立即数 $\rightarrow$ 寄存器、存贮器;

存贮单元 $\longleftrightarrow$ 寄存器。

不能是: 单元 $\longleftrightarrow$ 单元。

指令的具体形式可以为: **movb**、**movw**、**movl**

等





2. 数据交换指令

格式: XCHG OPS, OPD

功能: (OPD) $\rightarrow$ OPS, (OPS) $\rightarrow$ OPD。

如: xchg %eax, %ebx

若执行前: EAX = 0x5678, EBX = 0x1234

执行后: EAX = 0x1234, EBX = 0x5678H

注意:

寄存器 $\longleftrightarrow$ 寄存器, 寄存器 $\longleftrightarrow$ 存储器。有一个必须为寄存器。





3. 扩展传送指令

(1) 符号扩展传送指令

语句格式: **MOVSX** OPS, OPD

功能: 将源操作数的符号向前扩展成与目的操作数相同的数据类型后, 再送入目的地址对应的单元中。



(1) 符号扩展传送指令

说明:

- 可以指明操作数类型，具体指令可以是 **movsbw、movsbl、movswl**
- OPS 不能为立即数;
- OPD 必须是寄存器;
- 源操作数的位数必须小于目的操作数的位数



(2) 无符号扩展传送指令

语句格式: **MOVZX** OPS, OPD

功能: 将源操作数的高位补0, 扩成与目的操作数相同的数据类型后, 再送入目的地址对应的单元中。



(2) 无符号扩展传送指令

说明:

- 可以指明操作数类型，具体指令可以是 **movzbw**、**movzbl**、**movzwl**
- OPS 不能为立即数；
- OPD 必须是寄存器；
- 源操作数的位数必小于目的操作数的位数。



(3) 示例

例：

```
mov    $0xe3, %bl
```

```
movsx  %bl, %ebx
```

(EBX) = ?

0xFFFFFFFFE3

若将最后一条指令换成

```
movzx  %bl, %ebx
```

(EBX) = ?

0x000000E3

(3) 示例



华中科技大学

例:

byte0: .byte 0xa8

mov byte0, %bl

movsx %bl, %ecx # ERROR

movsx byte0, %bl # ERROR





4.地址传送指令

格式: **LEA** OPS, OPD

功能: OPS的偏移地址 → OPD。

如: **leal buf, %eax**

等价于: **movl \$buf, %eax**

注意: **OPD必须为寄存器。**

等价于**MOV \$OPS, OPD。**



二、算术运算指令

算术指令包括：加、减、乘、除及符号扩展指令

指令的共同特点：对SF、OF、ZF、CF、AF等标志有影响

运算原则：有符号数在机内均用补码表示，最高位为符号位，计算机在运算时，不单独处理符号，而是将符号作为数值一起参加运算。





1. 加法指令

- **ADD OPS, OPD**

功能: $(OPS) + (OPD) \rightarrow OPD$

- **INC OPD**

功能: $(OPD) + 1 \rightarrow OPD$

注意: OPD不能是立即数。



2.减法指令

- SUB OPS, OPD

功能: $(OPD) - (OPS) \rightarrow OPD$

- DEC OPD

功能: $(OPD) - 1 \rightarrow OPD$

- NEG OPD

功能: $(OPD) \text{ 反} + 1 \rightarrow OPD$ 。(求补)

- CMP OPD, OPS

功能: $(OPD) - (OPS)$, 不回送结果, 只影响标

志。





2.减法指令

注意：如果是进行加、减运算，对**有符号数**，当**OF=0**时，运算结果**正确**；对**无符号数**，当**CF=0**时，结果**正确**。



2.减法指令

例：求 (AX) 的绝对值。

```
cmp $0, %ax
```

```
jge exit ; AX ≥ 0时, 跳到exit
```

```
neg %ax
```

```
...
```

```
exit:
```

```
...
```



3.乘法指令

(1)单操作数乘法指令

有符号乘法: **IMUL OPS;**

无符号乘法: **MUL OPS;**

被乘数隐含在EAX/AX/AL中。是字乘法还是字节乘法，由OPS决定。





(1) 单操作数乘法指令

功能:

字节乘法: $(AL) * (OPS) \rightarrow AX$

字乘法: $(AX) * (OPS) \rightarrow DX, AX$

双字乘法: $(EAX) * (OPS) \rightarrow EDX, EAX$



(1) 单操作数乘法指令

如:

```
mov $0x50, %ax
```

```
mov $-0x10, %bx
```

```
imul %bx
```

结果: DX= FFFFH, AX= FB00H。



(1) 单操作数乘法指令

注意：

- 目的操作数必须是AX（字乘法是AX，字节是AL）。
- OPS不能是立即数。
- 除了对CF和OF有影响外，对其它标志的改变无意义。
- 若MUL运算后，(AH) 或 (DX) 为0，则CF、OF均为0；否则CF、OF均为1。
- 对IMUL来说，若乘积的高一半是底一半的符号扩展，则CF、OF=0；否则CF、OF=1。





(2) 双操作数乘法指令

格式: IMUL OPS, OPD

功能: $(OPS) * (OPD) \rightarrow OPD$

注意:

- 目的操作数必须是16/32位寄存器
- 源操作数可以是立即数。
- 目的操作数和源操作数必须类型一致



(3) 三操作数乘法指令

格式: IMUL n, OPS, OPD

功能: $(OPS) * n \rightarrow OPD$

注意:

- 目的操作数必须是16/32位寄存器
- 源操作数不能是立即数。
- 目的操作数和源操作数必须类型一致



4. 符号扩展指令

同乘法一样，除法也有字和字节之分。如果是字除法，被除数也要求是双精度数。从单精度数到双精度数，涉及到符号扩展的问题。

补码的符号位扩展：8位补码 $(-1) = \text{FFH}$ ，16位补码 $(-1) = \text{FFFFH}$ 。





(1) 字节转换成字

格式: CBW

功能: 将AL中的符号扩展到AH中。

如:

```
mov $-7, %al
```

```
cbw
```

CBW执行前: (AL) = F9H,

执行后: (AX) = FFF9H。



华中科技大学

(2) 字转为双字

格式：CWD

功能：将AX的符号扩展到DX中。





(3) 字转为双字

格式：CWDE

功能：将AX的符号扩展到EAX中。

(4)32位转为64位



华中科技大学

格式：CDQ

功能：将EAX的符号扩展到EDX中。





5. 除法指令

有符号除法: IDIV OPS

无符号除法: DIV OPS

功能:

字节除法: $(AX) / (OPS) \rightarrow AL$ (商)、 AH (余数)

字除法: $(DX, AX) / (OPS) \rightarrow AX$ (商)、 DX (余)

双字除法: $(EDX, EAX) / (OPS) \rightarrow EAX$ (商)、 EDX (余)

由OPS决定是字节、字除法。





5. 除法指令

例:

```
mov $-0x4001, %ax
```

```
cwd
```

```
mov $4, %cx
```

```
idiv %cx
```

结果: (DX) = FFFFH (余数) , (AX
) = F000H (商) 。



5. 除法指令

例:

```
mov $-0x4001, %ax
```

```
cwd
```

```
mov $-0x4, %cx
```

```
idiv %cx
```

结果: (DX) = FFFFH (余数) , (AX)
= 1000H (商) 。





5. 除法指令

注意:

- 如果是无符号除法，被除数符号的扩展不能用 CBW、CWD。只能：
 MOV AX, A;
 MOV DX, 0.
- OPS不能为立即数。
- 除数为0时，产生溢出，导致溢出中断。
- 有符号除法，余数与被除数符号相同。



三、按位运算指令

1. 逻辑运算指令

包括：求反、逻辑乘、测试、逻辑加、按位加等。



(1) 求反

格式：NOT OPD;

功能：将OPD的内容逐位取反→OPD。

该指令不影响标志位。

注意：与求补的区别。（求补NEG OPD）。

例：

not %ah

执行前：AH=20H，执行后：AH=DFH。



(2) 逻辑乘

格式: **AND OPD, OPS;**

功能: **$OPD \wedge OPS \rightarrow OPD$**

该指令对**SF、ZF、PF、OF、CF**有影响。

运算规则: **$1 \wedge 1 = 1, 1 \wedge 0 = 0, 0 \wedge 1 = 0, 0 \wedge 0 = 0$** 。

例:

and \$0xff, %dx

执行前: **(DX) = ABCDH**, 执行后: **(DX) = CDH**。屏蔽高8位。





(2) 逻辑乘

例：

and 0x0f, %al

执行前：(AL) = '5' = 35H, 执行后：(AL) = 5, 得到 '5' 字符的实际值5。





(2) 逻辑乘

AND也可作为常量表达式运算符:

例:

```
.equ a1, 0xb6
```

```
and a1 and $0xfd, %a1
```

在汇编过程中, 将式子A AND 0FDH的值求出为:
0B4H。





(3) 测试指令

a) 格式: TEST OPD, OPS

功能: $(OPD) \wedge (OPS)$, 结果不回送, 影响标志SF、ZF、PF。

用途: 检测与OPS中为1的位相对应的位是否为1。





(3) 测试指令

例:

```
test $0x80, %al
```

```
jnz h1
```

```
...
```

```
h1:
```

```
mov %cx, %bx
```

```
...
```

测试al最高位是否为0，不为0则跳转到h1处



(3) 测试指令

b) 格式: BT OPD, OPS

功能: 将OPD的指定位送到CF

注意:

- **OPD为16/32位**
- **OPS为立即数或寄存器**
- **若OPS绝对值大于OPD位数, 则取模**



(4) 逻辑加

例:

or \$0x55, %ah

OR AH, 55H

**执行前: (AH) = 0AAH, 执行后: (AH)
= 0FFH**



(4) 逻辑加

格式: OR OPD, OPS

功能: $(OPD) \vee (OPS) \rightarrow OPD$,

影响标志位: CF、OF、PF、SF、ZF。

运算法则: $1 \vee 1 = 1$, $1 \vee 0 = 1$, $0 \vee 1 = 1$, $0 \vee 0 = 0$ 。



(5) 按位加

格式: XOR OPD, OPS

功能: (OPD) (OPS) $\rightarrow$ OPD,

影响标志: CF、OF、PF、SF、ZF。

运算法则: $1 \oplus 1 = 0$, $1 \oplus 0 = 1$, $0 \oplus 1 = 1$, $0 \oplus 0 = 0$ 。





(5)按位加

例:

```
xor %ax, %ax
```

执行后: (AX) =0, 等价于MOV AX, 0



(5)按位加

例:

```
xor $0x123, %ax
```

```
jz h1
```

```
...
```

```
h1:
```

```
mov $5, %bx
```

```
...
```

若AX=123H, 则跳转到h1



2. 移位指令

包括：算术、逻辑、循环移位。

格式：

[标号:] 操作符 n, OPD

[标号:] 操作符 %cl, OPD



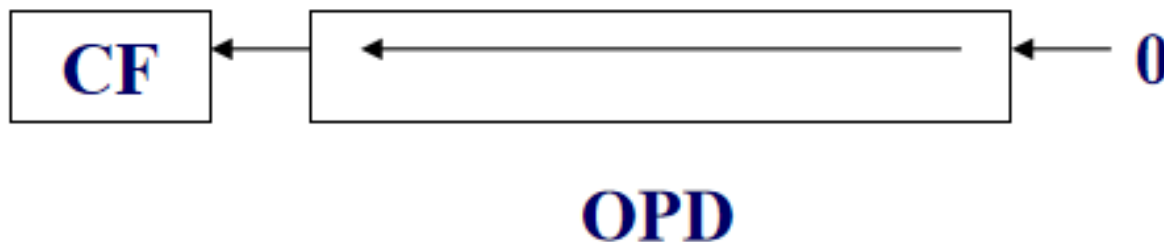
2. 移位指令

a) 算术左移或逻辑左移

SAL n, OPD, 或 SHL n, OPD

功能： (OPD) 向左移指定的次数，低位补0。

每左移一次，相当于原来的数*2，左移n次，相当于 $*2^n$ 。





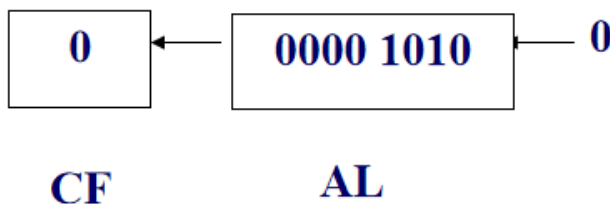
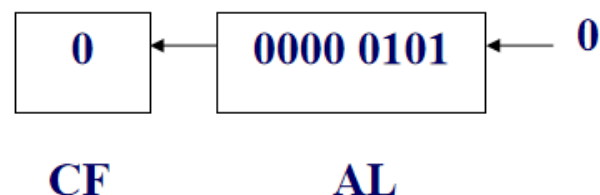
a) 算术左移或逻辑左移

例：

`sal $1, %al`

执行前：AL=5,

执行后：AL=0AH



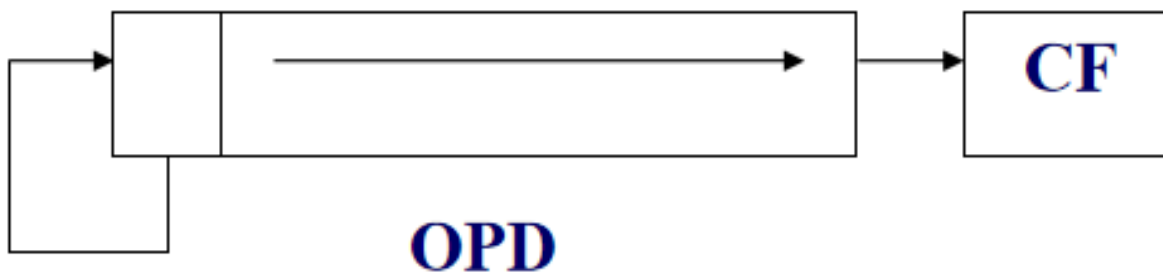


b) 算术右移

SAR n, OPD

功能：（OPD）向右移指定位数，最高位不变。

右移n位，实现有符号数除 2^n 运算。



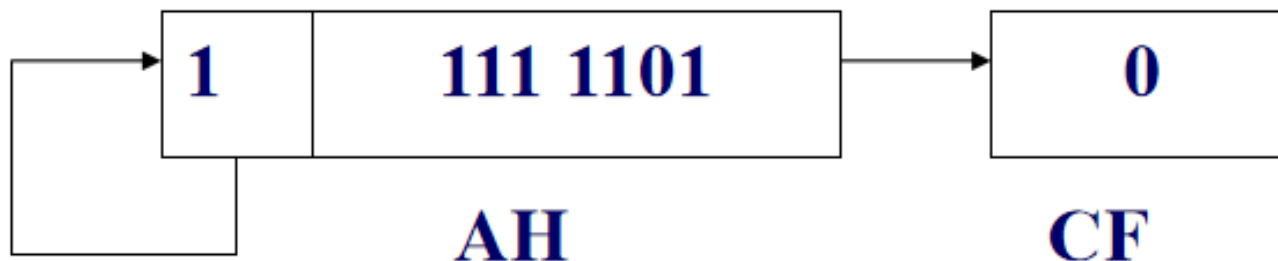
b) 算术右移

例：

`sar $2, %ah`

执行前：AH= 0F4H,

执行后：AH= 0FDH。



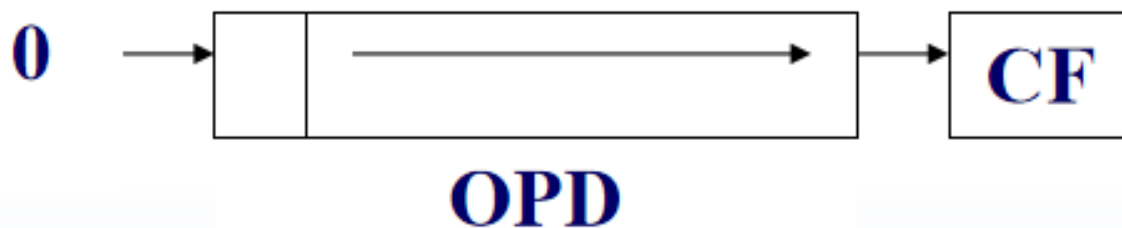


c) 逻辑右移

SHR OPD, n

功能： (OPD) 向右移指定位数，最高位补相应个数的0。

右移n位，实现无符号数除 2^n 运算。





c) 逻辑右移

例：将AL中的压缩BCD码转换为非压缩BCD码。

...

```
mov %al, %ah
```

```
shr $4, %ah ; 高位的BCD码变成8位
```

```
and $0x0f, %al ; 低位的BCD码变成8位
```

...

结果在(AH)、(AL)中。



c) 逻辑右移

注意：

- **算术移位适合于有符号数；逻辑移位适合于无符号数。**



c) 逻辑右移

注意：

- **算术移位适合于有符号数；逻辑移位适合于无符号数。**

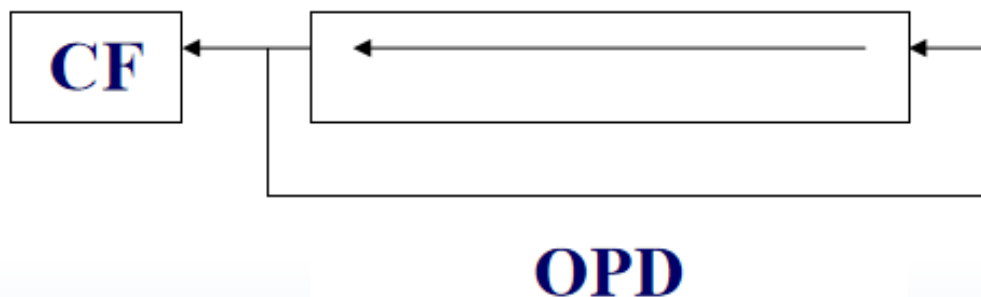


(2) 循环移位指令

a) 循环左移

ROL n, OPD

功能：（OPD）的最高位与最低位连成一个环，移位。

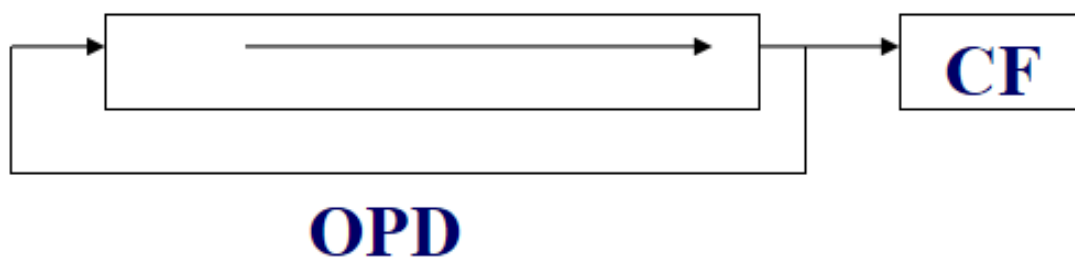




b) 循环右移

ROR OPD, n

功能： (OPD) 的最低位与最高位连成一个环， 移位。



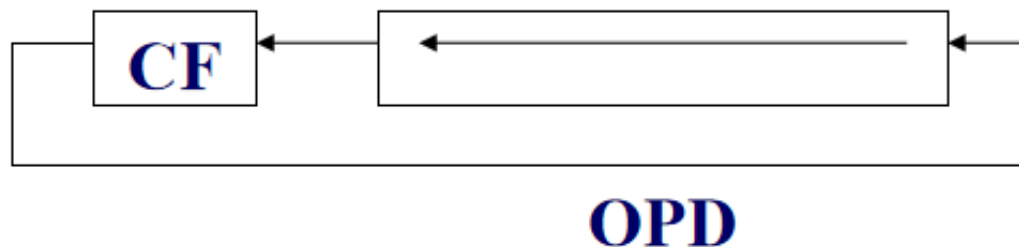


c) 带进位循环左移

RCL OPD, n

功能： (OPD) 连同CF一起向左循环移动规定次数

。



c) 帶進位循環左移

例：

將AX的高4位移到DX的低4位。

```
mov $4, %cx
```

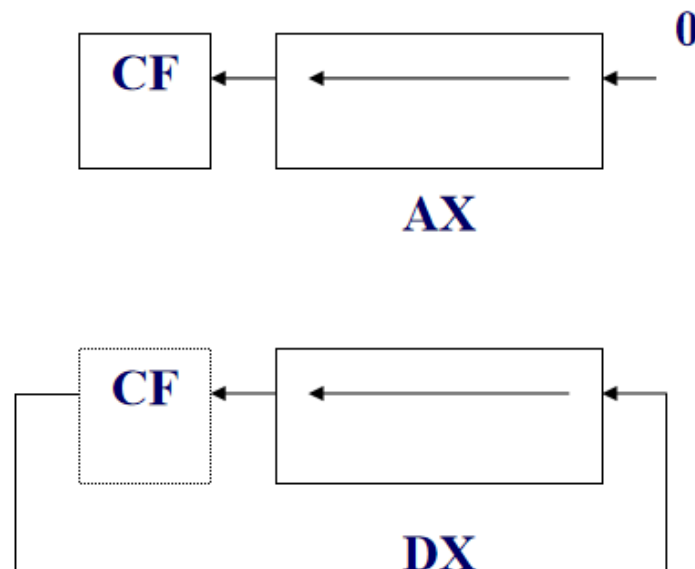
```
next:
```

```
sal $1, %ax
```

```
rcl $1, %dx
```

```
dec %cx
```

```
jne next
```

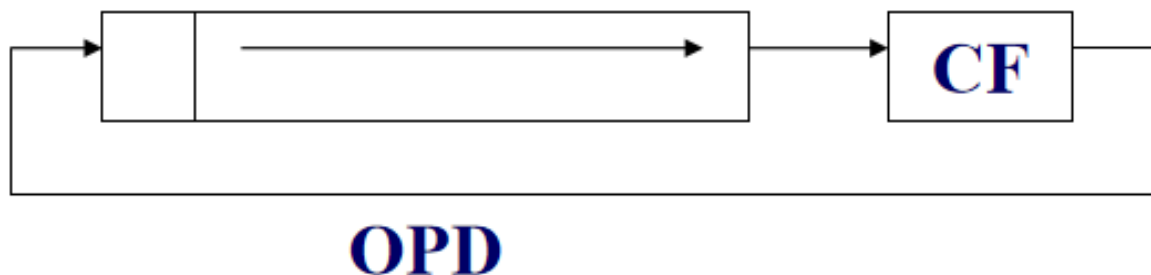




d) 带进位循环右移

RCR n, OPD

功能： (OPD) 的连同CF一起向右循环移动规定次数。



2. 移位指令



华中科技大学

移位指令格式小结:

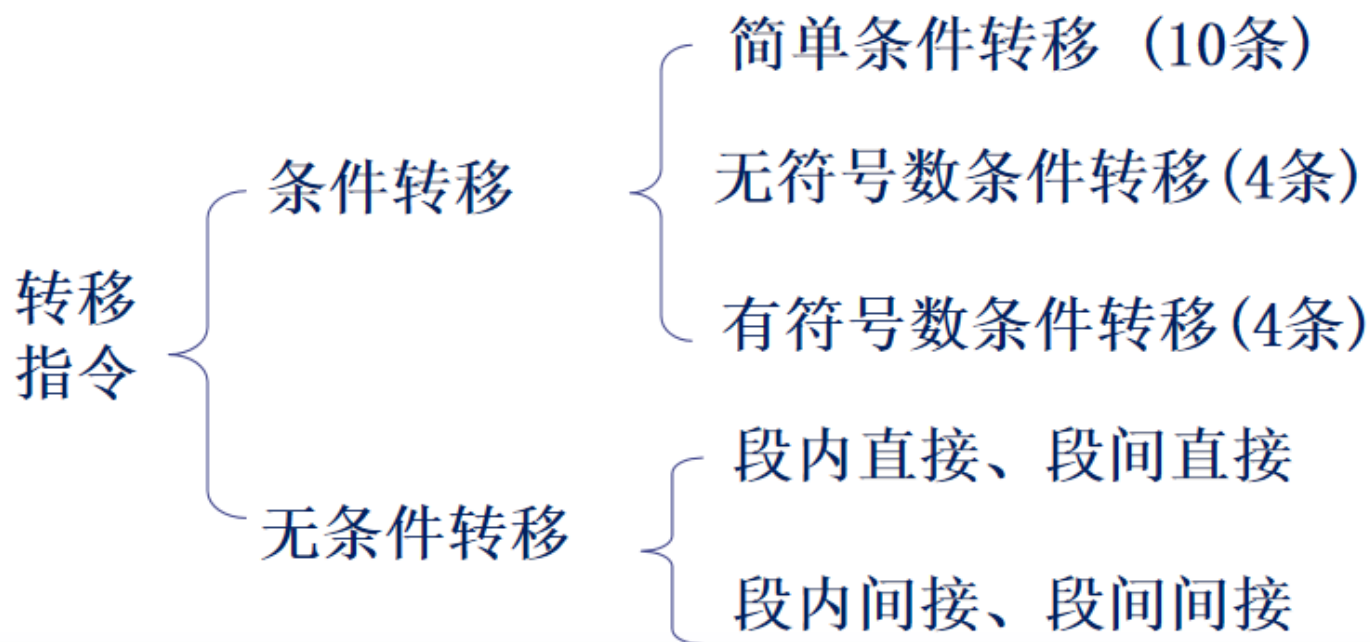
S	A	L
		R
	H	L
		R
R	O	L
		R
	C	L
		R





四、控制转移指令

特点：改变程序的执行顺序，即改变了指令指示器IP的内容。





四、控制转移指令

转移指令的本质，是修改IP寄存器的内容：

- 将目的地址送到IP寄存器中
- 下一条指令则跳转到目的地址开始执行

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program:...
```

CS:0000	B8D40A	mov	ax,0AD4	ax	0001	c=0
CS:0003	8ED8	mov	ds,ax	bx	0003	z=0
CS:0005	B93200	mov	cx,0032	cx	0031	s=0
CS:0008	B80000	mov	ax,0000	dx	0000	o=0
CS:000B	BB0100	mov	bx,0001	si	0000	p=0
CS:000E	03C3	add	ax,bx	di	0000	a=0
CS:0010	43	inc	bx	bp	0000	i=1
CS:0011	43	inc	bx	sp	00C8	d=0
CS:0012	49	dec	cx	ds	0AD4	
CS:0013	75F9	jne	000E ↑	es	0AB7	
CS:0015	A30000	mov	[0000],ax	ss	0AC7	
CS:0018	B44C	mov	ah,4C	cs	0AD5	
CS:001A	CD21	int	21	ip	0013	
CS:001C	3A06C226	cmp	al,[26C2]			
CS:0020	750B	jne	002D			

```
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program:...
```

CS:0000	B8D40A	mov	ax,0AD4	ax	0001	c=0
CS:0003	8ED8	mov	ds,ax	bx	0003	z=0
CS:0005	B93200	mov	cx,0032	cx	0031	s=0
CS:0008	B80000	mov	ax,0000	dx	0000	o=0
CS:000B	BB0100	mov	bx,0001	si	0000	p=0
CS:000E	03C3	add	ax,bx	di	0000	a=0
CS:0010	43	inc	bx	bp	0000	i=1
CS:0011	43	inc	bx	sp	00C8	d=0
CS:0012	49	dec	cx	ds	0AD4	
CS:0013	75F9	jne	000E	es	0AB7	
CS:0015	A30000	mov	[0000],ax	ss	0AC7	
CS:0018	B44C	mov	ah,4C	cs	0AD5	
CS:001A	CD21	int	21	ip	000E	
CS:001C	3A06C226	cmp	al,[26C2]			
CS:0020	750B	jne	002D			

```
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```





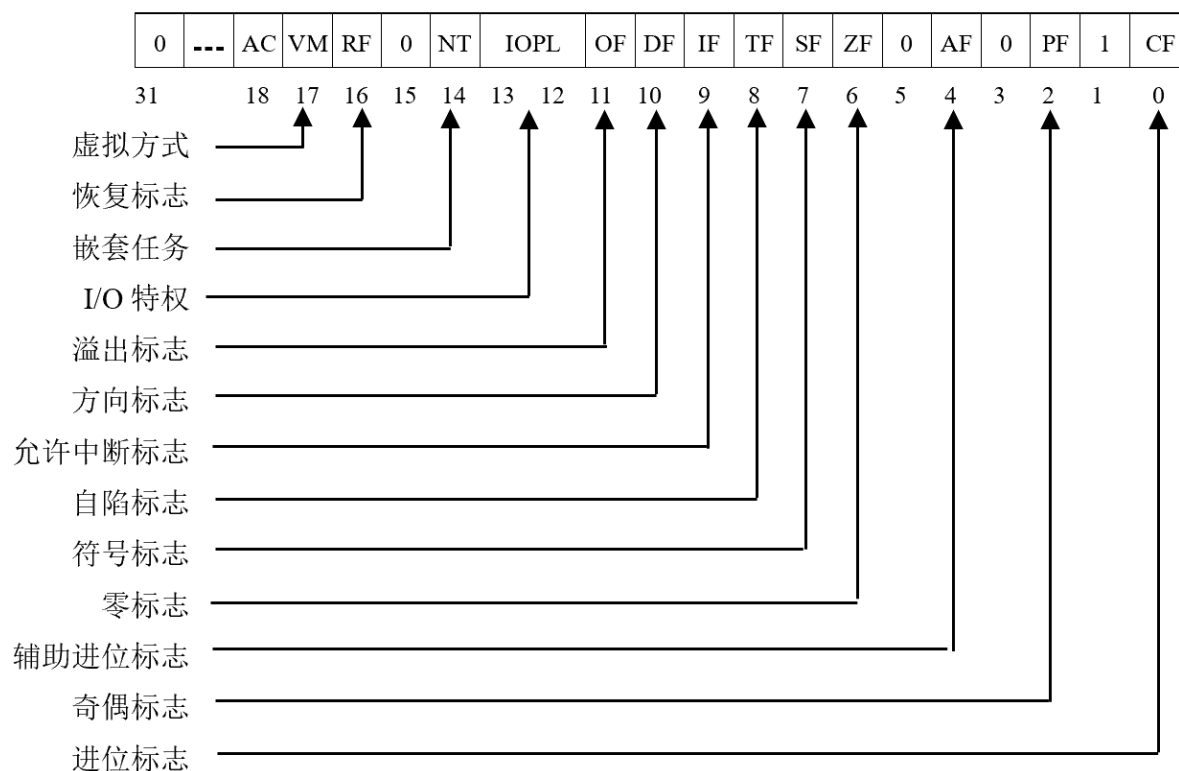
1. 标志寄存器

保存一条指令执行之后，CPU所处状态的信息及运算结果的特征。32标志寄存器称为EFLAGS（也称为程序状态字寄存器（PSW）。



1. 标志寄存器

保存一条指令执行之后，CPU所处状态的信息及运算结果的特征。32标志寄存器称为EFLAGS（也称为程序状态字寄存器（PSW））。



(1) 符号标志SF (Sign Flag)



华中科技大学

运算结果的最高二进制位为1，则 $SF=1$ ，否则

$SF=0$



(2) 进位标志 CF (Carry Flag)



华中科技大学

运算时从最高位向前产生了进位（或借位），则
 $CF=1$ ；否则 $CF=0$ 。





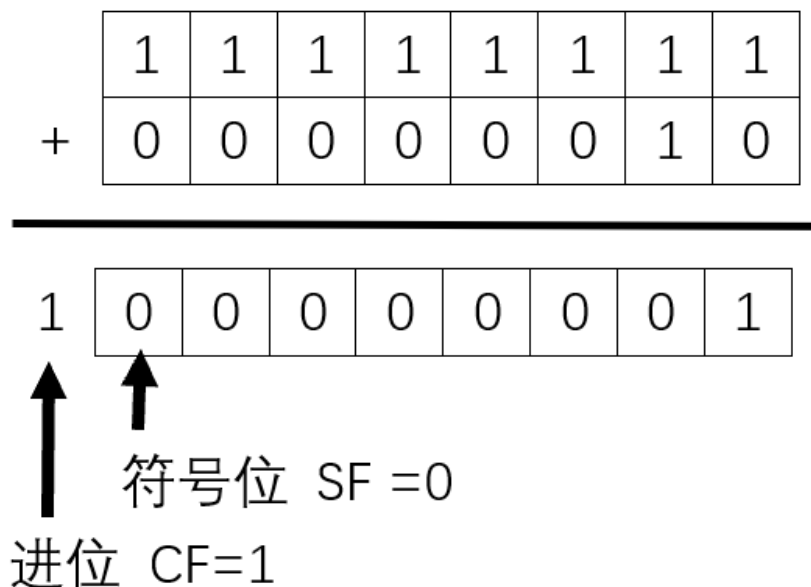
(2) 进位标志 CF (Carry Flag)

例:

```
mov $0xff, %ah
```

```
add $2, %ah
```

执行后: AH = 1



(3) 零标志 ZF (Zero Flag)



华中科技大学

运算结果为0, 则 $ZF = 1$, 否则 $ZF = 0$



(4) 溢出标志 OF (Overflow Flag)



华中科技大学

运算结果超出了有符号数的范围，则 $OF=1$ 。





(4) 溢出标志 OF (Overflow Flag)

例:

```
mov $0xff, %ah
```

```
add $2, %ah
```

执行后: AH = 1

CF=1, SF=0, ZF=0

OF=0



(4) 溢出标志 OF (Overflow Flag)



华中科技大学

溢出标志硬件层的判断方法：

当出现“正+正=负、负+负=正、正-负=负、负-正=正”，产生溢出”

例如，执行加法时，两个加数的最高位相同，且加的结果的最高位与加数最高位相反，则 $OF=1$ 。

两个加数的最高位不同，则 $OF = 0$





2. 条件转移

功能：由上一条指令所设的条件码来判别测试条件，满足条件则转移到指令所指地址去执行，否则循环执行。

满足条件时，当前 (IP) + 符号扩展到16位的位移量 $\rightarrow$ IP。位移范围在 -128 ~ 127 之间。





(1) 简单条件转移

根据单个标志位 CF、ZF、SF、OF、PF 的值确定是否转移。

格式： [标号:] 操作符 短标号



(1) 简单条件转移

a) CF标志: JC / JNC

JC表示: $CF=1$ 时转移。例如两数相减, 低于。

JNC表示: $CF=0$ 时转移。例如两数相减, 高于或等于。

b) ZF标志: JZ / JNZ

JZ表示: 结果为0, 转移, 此时 $ZF=1$ 。

JNZ表示: 结果不为0, 转移, 此时 $ZF=0$ 。

c) SF标志: JS / JNS

JS表示: 结果为负, 转移, 此时 $SF=1$ 。

JNZ表示: 结果不为正, 转移, 此时 $SF=0$ 。



(1) 简单条件转移

d) OF标志: JO / JNO

JO表示: 结果溢出, 转移, 此时OF=1。

JNO表示: 结果不溢出, 转移, 此时OF=0。

e) PF标志: JP / JNP或JPE / JPO

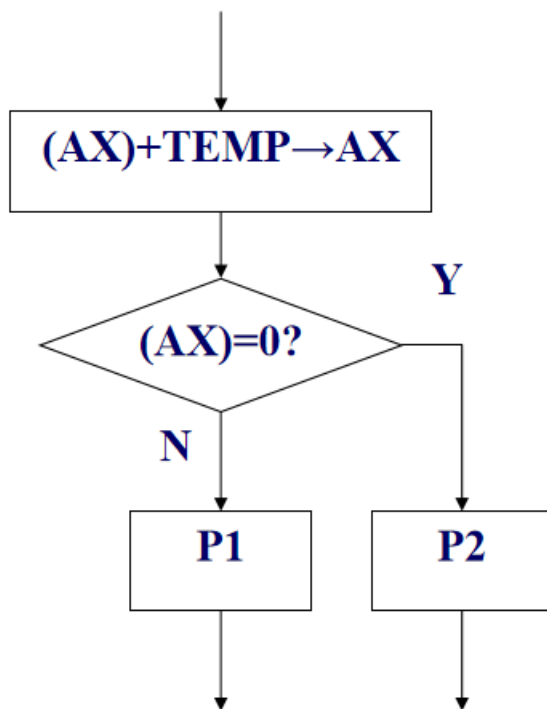
JP表示: 低8位中1的个数为偶数时转移, 此时OF=1。

JNP表示: 低8位中1的个数为奇数时转移, 此时OF=0。



(1) 简单条件转移

例：两数相加，结果为0，则转移到P2，否则运行P1。



```
...  
    add temp,  
    %ax  
    jz p2  
    ... ; p1  
p2:  
...
```



華中科技大學

(2) 无符号数的条件转移

此指令跟在比较指令之后，比较的对象为无符号数。

结果有：高于、高于等于、低于、低于等于。

Above: 高于, below: 低于, equal: 等于。





(2) 无符号数的条件转移

a) JA / JNBE

JA表示：高于则转移。

JNBE表示：不低于或等于转移。

测试条件： $CF \vee ZF = 0$ 。

(分析： $a-b \geq 0$ ，此时 $CF=0$ ； $a-b \neq 0$ ，此时 $ZF=0$ 。)

b) JAE / JNB

JAE表示：高于等于则转移。

JNB表示：不低于则转移。

测试条件： $CF=0$ 或 $ZF=1$ 。





(2) 无符号数的条件转移

c) JB / JNAE

JB表示：低于则转移。

JNAE表示：不高于且不等于则转移。

测试条件：CF=1。

d) JBE / JNA

JBE表示：低于等于则转移。

JNA表示：不高于则转移。

测试条件：CF $\vee$ ZF=1。



(3)有符号数条件转移

比较结果分为4种：大于、大于等于、小于、小于等于。

Great: 大于, Little: 小于, Equal: 等于。



(3)有符号数条件转移

a) JG / JNLE

JG表示：两数相比，大于，则转移。

JNLE表示：两数相比，不小于且不等于，则转移

。

测试条件： $(SFOF) \vee ZF=0$ 。

b) JGE / JNL

JGE表示：两数相比，大于等于，则转移。

JNL表示：两数相比，不小于，则转移。

测试条件： $SFOF=0$ 。





(3)有符号数条件转移

c) JL / JNGE

JL表示：两数相比，小于，则转移。

JNGE表示：两数相比，不大于且不等于，则转移

。

测试条件：SFOF=1。

d) JLE / JNG

JLE表示：两数相比，小于等于，则转移。

JNG表示：两数相比，不大于，则转移。

测试条件：(SFOF) $\vee$ ZF=1。





(3)有符号数条件转移

使用转移指令时应注意：

CMP比较指令本身无法分别有、无符号数，它比较的是否有符号，由后面的转移指令确定。

例：

```
mov  $-0x40, %al
```

```
cmp  $0x50, %al
```

```
jg  l1 ; 比较的是有符号数， (-40H) < 50H， 不转移
```

```
...
```

```
l1:
```

```
...
```





(3)有符号数条件转移

例:

```
mov $-0x40, %al
```

```
cmp $0x50, %al
```

```
jg l1 ; 比较的是有符号数,  $(-40H) < 50H$ , 不转移
```

...

```
l1:
```

...

若将JG换为JA, 就变成无符号数了, 此时, $(AL) = (-40H)$
 $(-40)补 = C0H > 50H$, 转移。

3.无条件转移指令



华中科技大学

功能：无条件地转移到目的地址执行。

格式：JMP OPD





(1)按目标地址的转移范围划分

a) 段内转移

执行后只改变eip的内容

b) 段间转移

执行后改变了cs和eip的内容

注：32位保护模式下，不允许应用程序进行段间
跳转



华中科技大学

(2)按目标地址的寻址方式划分

a) 直接转移

OPD为标号，或地址值





a) 直接转移

例:

jmp label_1 ; label_1为代码段中的一个标号。跳转到label_1处

**jmp 0x8048377 ; label_1的地址为0x8048377
， 则也跳转到label_1处**



a) 直接转移

注：转移指令中的位移量，是指目的地址到转移指令下一条指令的字节距离。当位移量在-128 ~ 127之间时，转移称为短转移。否则段称为近转移。

汇编程序会根据位移量大小自动形成短转移或近转移指令，32位保护模式下，不需要在汇编语句中区分。



b) 间接转移

OPD为寄存器操作数，或内存操作数。JMP以OPD的内容作为转移目的地址。



b) 间接转移

例:

`mov $label_1, %eax` ; 将标号label_1的地址传送到EAX中

`jmp * %eax` ; *表示是间接跳转。OPD为寄存器操作

数。由于EAX的内容为label_1的地址, 所以跳转到label_1处

例:

`movl $label_1, var_1` ; var_1是数据段中定义的一个变量。将label_1的地址传送到var_1中

`jmp * var_1` ; *表示是间接跳转。OPD是内存操作数

。由于var_1的内容为label_1的地址, 所以跳转到label_1处





b) 间接转移

例:

```
movl $label_1, var_1
```

```
mov    $var_1, %eax           ; 将变量var_1的地址传送到EAX中
```

```
jmp    * (%eax)              ; *表示是间接跳转。OPD是内存
```

操作数。由于(%eax)的内容是label_1的地址，所以跳转到label_1处

注:

jmp * %eax和jmp * (%eax)，都是间接跳转。但目标操作数的不同，前者为寄存器操作数，后者为内存操作数。所以两者的执行效果也不同。



b) 间接转移

例：

根据不同的输入，执行不同的程序片段。

输入1，执行程序段 LP1；

输入2，执行程序段 LP2；

输入3，执行程序段 LP3；



b) 间接转移

直接跳转实现：

...

JMP LP1

...

JMP LP2

...

JMP LP3

评：如果分支很多，每个分支均使用 **JMP** 标号，程序难看，
臃肿！



b) 间接转移

间接接跳转实现:

; 构造指令地址列表, 采用间接转移指令

functab: .long lp1, lp2, lp3, ...

...

jmp functab(, %ebx, 4); EBX = 0, 跳转到lp1处。EBX =
1, 跳转到lp2处



4. 转移传送指令

带条件的数据传送指令

格式: `cmov*** OPS, OPD`

功能: 在条件成立时, 传送数据,



4. 转移传送指令

注意：

- OPS为寄存器操作数或内存操作数
- OPD为寄存器操作数
- 可使用单个标志位 ZF、CF、F、OF、PF的值确定是否传送，相应指令为：cmovne/cmovnz、cmovnc、cmovns、cmovno、cmovnp
- 可使用比较结果：高于、高于等于、低于、低于、大于、大于等于、小于、小于等于确定是否传送，相应指令为：cmova、cmovae、cmovb、cmovbe、cmovg、cmovge、cmovl、cmovle



五、分支

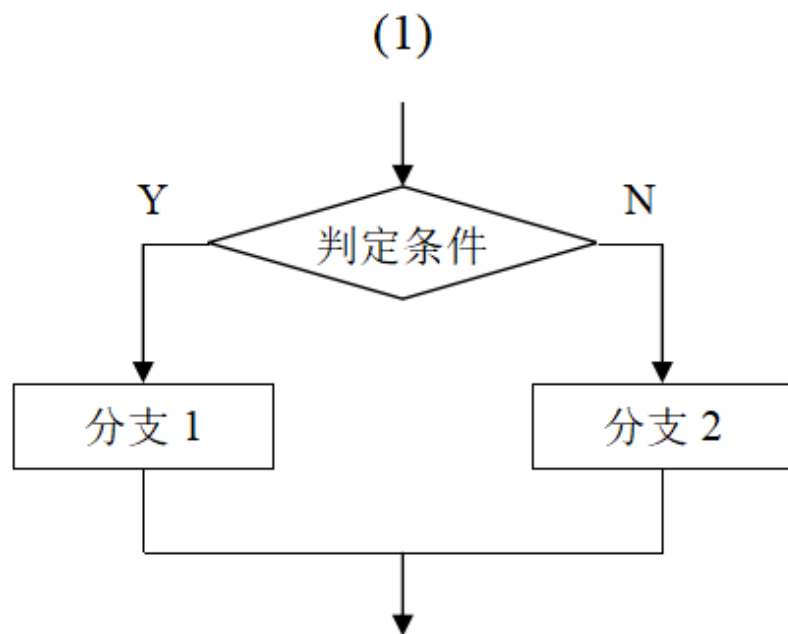


华中科技大学

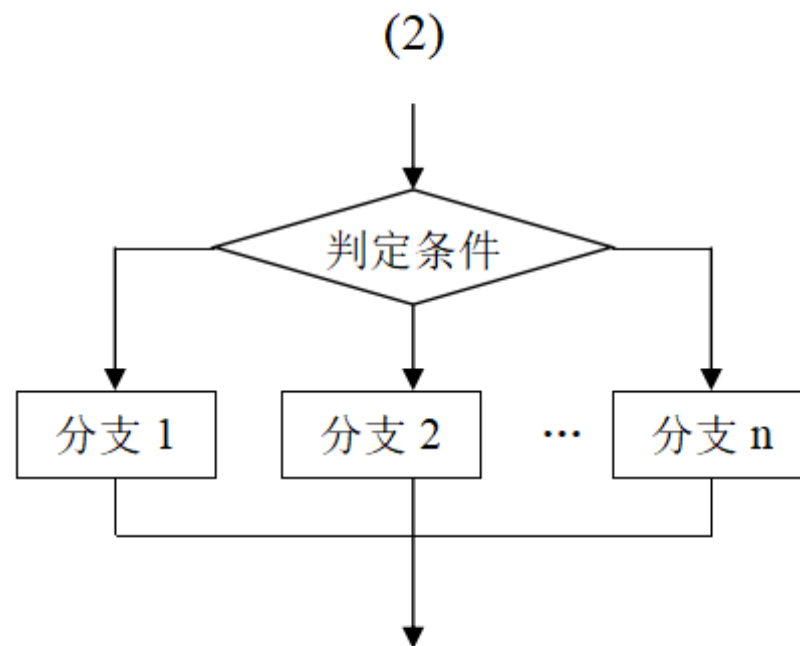
特点：计算机根据不同情况自动作出判断，有选择地执行相应处理程序。



1. 两种结构形式



相当于 C 语言的 if、else



相当于 C 语言的 switch()

结构特点：运行方向是向前的，条件确定，只能执行分支中的一个。

2.程序设计方法



华中科技大学

程序分支一般用条件转移指令产生，不同条件是通过EFLAGS的标志状态反映出来。

注：转移指令不影标志状态。



2.程序设计方法



华中科技大学

例：

比较数组BUF中的三个16位补码，若三个数都不相等则显示0，有两个相等则显示1，都相等则显示2。

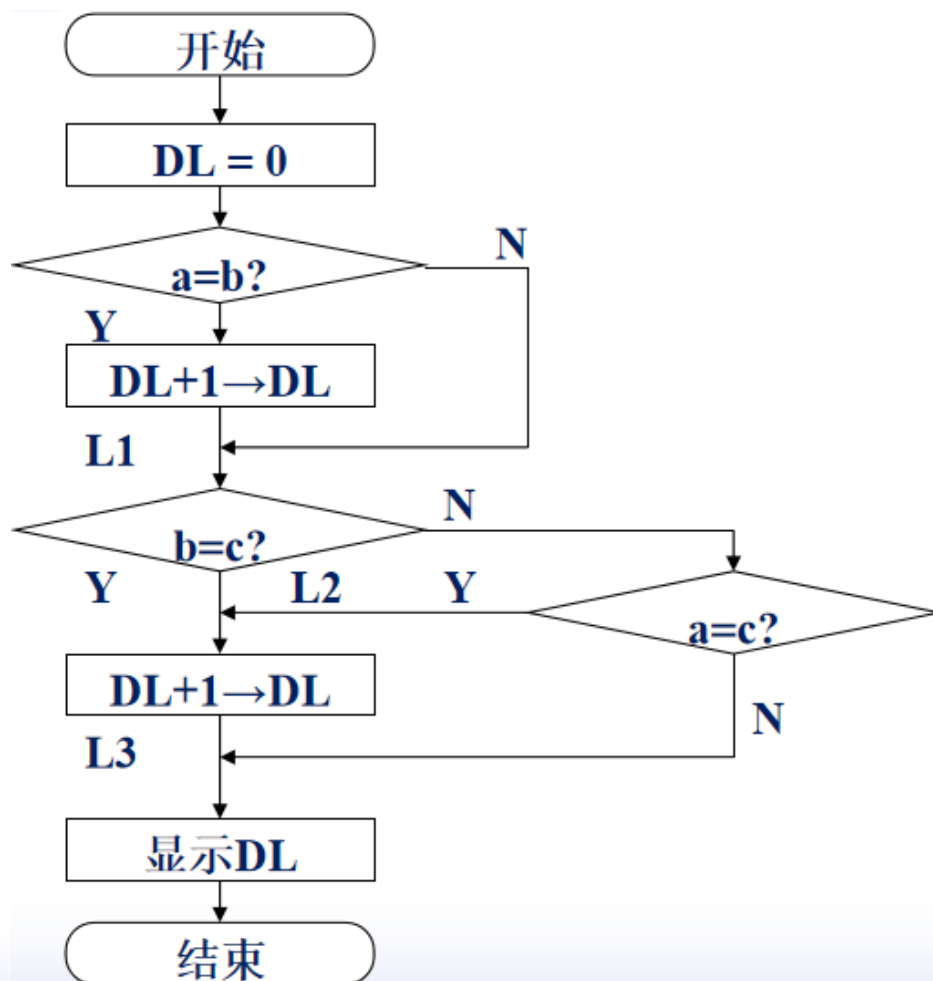
分析：假定三个数为a、b、c，D为比较结果。我们可以画出流程图：



2.程序设计方法



华中科技大学



2.程序设计方法



华中科技大学

```
# 3_number.s
.section .data
    buf: .word 12, 12, 12
    r1: .byte 0, '\n'
.section .text
.global _start
_start:
    mov $0, %dl
    mov buf, %ax
    cmp buf + 2, %ax
    jnz l1
    inc %dl
l1:
    mov buf + 2, %bx
    cmp buf + 4, %bx
    jz l2
```

```
    cmp buf + 4, %ax
    jnz l3
l2:
    inc %dl
l3:
    add $0x30, %dl
; DL变为ASCII码
    mov %dl, r1
    mov $4, %eax; 输出结果
    mov $1, %ebx
    mov $r1, %ecx
    mov $2, %edx
    int $0x80
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```





分支程序设计应注意

- 选择合适的转移指令

如：

```
cmp    $1, %ax
```

```
jl     label_1
```

JL为有符号转移指令， $(AX) < 1$ 则转移。若JL换为JB，则为无符号转移指令，该转移的反而不转移。





分支程序设计应注意

- 为每个分支安排出口

如：实现 $(AL) \geq 0, '+' \rightarrow DL;$

$(AL) < 0, '-' \rightarrow DL;$

```
cmp    $0, %al
```

```
jge    label_1
```

```
mov     '-', %dl
```

```
jmp     exit
```

```
label_1:
```

```
mov     '+', %dl
```

```
exit:
```

```
...
```



分支程序设计应注意

- **按流程图编程**

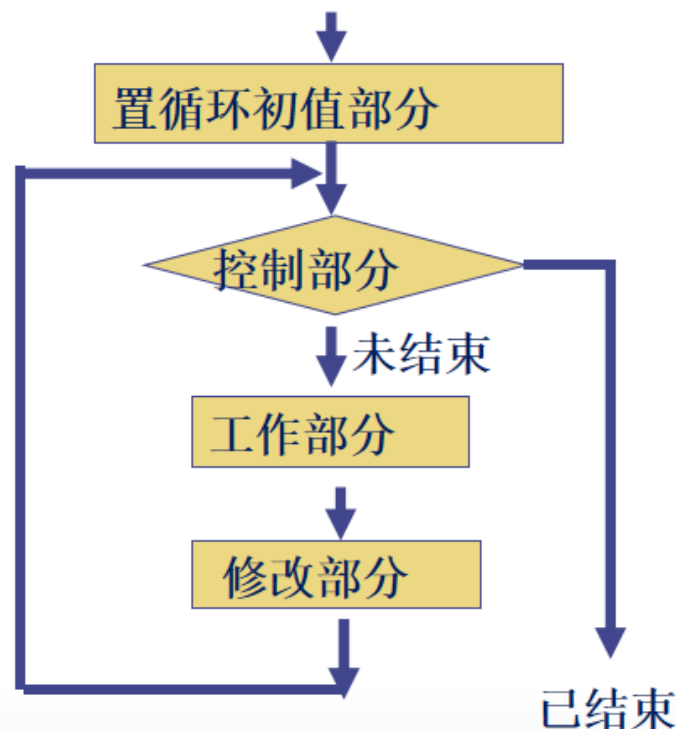
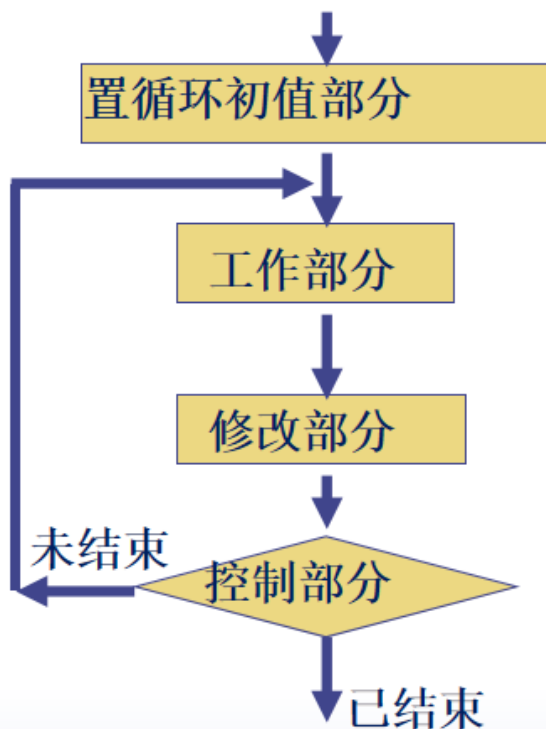
分支先后次序应尽量与问题提出的先后次序一致。

- **假定各种输入，调试分支程序。**

六、循环

1. 循环程序的结构

循环程序有两种结构形式：





1. 循环程序的结构

上述两种结构的循环程序，由三个部分组成：

- 设置循环的初始状态

置循环次数计数值以及循环体正常运行的初始状态值。

- 循环体

由循环的工作部分和修改部分组成。其中，工作部分完成程序功能；修改部分保证循环时，参加执行运行的信息发生规律性变化。

- 循环控制部分

保证循环程序按规定的次数或特定条件正常循环。





2. 循环控制方法

(1) 计数控制

特点：用于循环次数已知的情况。

a) 正计数法

特点：计数值开始为0，每次循环加1，直到加到n为止。





a) 正计数法

例:

```
    mov    $0, %ecx           ; 初始化
    ...
label_1:
    ...                       ; 循环体
    inc    %ecx               ; 控制部分
    cmp    n, %ecx
    jne    label_1
    ...
```



b) 倒数法

例:

```
    mov  n, %ecx           ; 初始化
    ...
label_1:
    ...                   ; 循环体
    dec  %ecx              ; 控制部分
    jnz  label_1
    ...
```

注: 其中dec %ecx; jnz label_1可以用指令loop
label1代替





b) 倒数法

例:

```
    mov  -n, %ecx           ; 初始化
    ...
label_1:
    ...                     ; 循环体
    inc  %ecx               ; 控制部分
    jnz  label_1
    ...
```



(2) 条件控制

特点：循环次数事先不知道。

例：

求AX中1的个数。

一般方法，逐位比较。



(2) 条件控制

一般方法，逐位比较。

1_count.s

...

mov \$0, %bl

; BL中存放1的个数

mov \$16, %cl

label_1:

sal \$1, %ax

; 算数左移, b15 -> CF

jnc label_2

inc %bl

; 若CF = 1, BL + 1 -> BL

label_2:

dec %cl

jnz label_1

上述方法必须循环16次。





(2) 条件控制

第二种方法不用循环16次就可以求得1的个数。

```
...  
    mov    $0, %cl          ; CL中存放1的个数  
label_1:  
    and    %ax, %ax         ; 产生条件  
    jz      exit  
    sal    $1, %ax           ; 算术左移, b15 -> CF  
    jnc     label_1  
    inc    %cl               ; 若CF = 1, CL + 1 -> CL  
    jmp     label_1  
exit:  
...
```





(3) 循环转移指令

a) LOOP 标号

功能: $(CX / ECX) - 1 \rightarrow CX / ECX$

若 (CX / ECX) 不为 0, 则转标号处执行。

基本等价于: `DEC CX / ECX`

`JNZ` 标号

注意: LOOP指令对标志位无影响!



b) LOOPE / LOOPZ 标号

功能: $(CX / ECX) - 1 \rightarrow CX / ECX$

若 (CX / ECX) 不为 0, 且 $ZF=1$, 则转标号处执行。

注意: 等于或为 0 循环转移指令, 本指令对标志位无影响。



b) LOOPE / LOOPZ 标号



华中科技大学

例:

```
mov $10, %cx  
mov $buf - 1, %bx
```

label_1:

```
inc    %bx  
cmpb $0, (%bx)  
loope label_1
```



c) LOOPNE/LOOPNZ 标号



华中科技大学

功能: $(CX / ECX) - 1 \rightarrow CX / ECX$

若 $CX / ECX \neq 0$, 且 $ZF = 0$, 则转标号处执行



d) JCXZ 标号 / JECXZ 标号



华中科技大学

功能：若 (CX / ECX)为0，则转标号处执行。

**先判断，后执行循环体时，可用此语句，标号为循环
结束处**





3. 循环程序设计

(1) 单重循环程序设计

特点：循环体内不再包含循环结构。

设计：

- 构思好算法
- 用C或者伪代码表达算法思想
- 理清算法和变量空间分配
- 分配好寄存器的用途，建立与变量的对应关系
- 按循环语句的执行过程，写出汇编语言程序





a) 循环次数已知

对于循环次数已知的情况，通常采用计数控制方法实现循环。

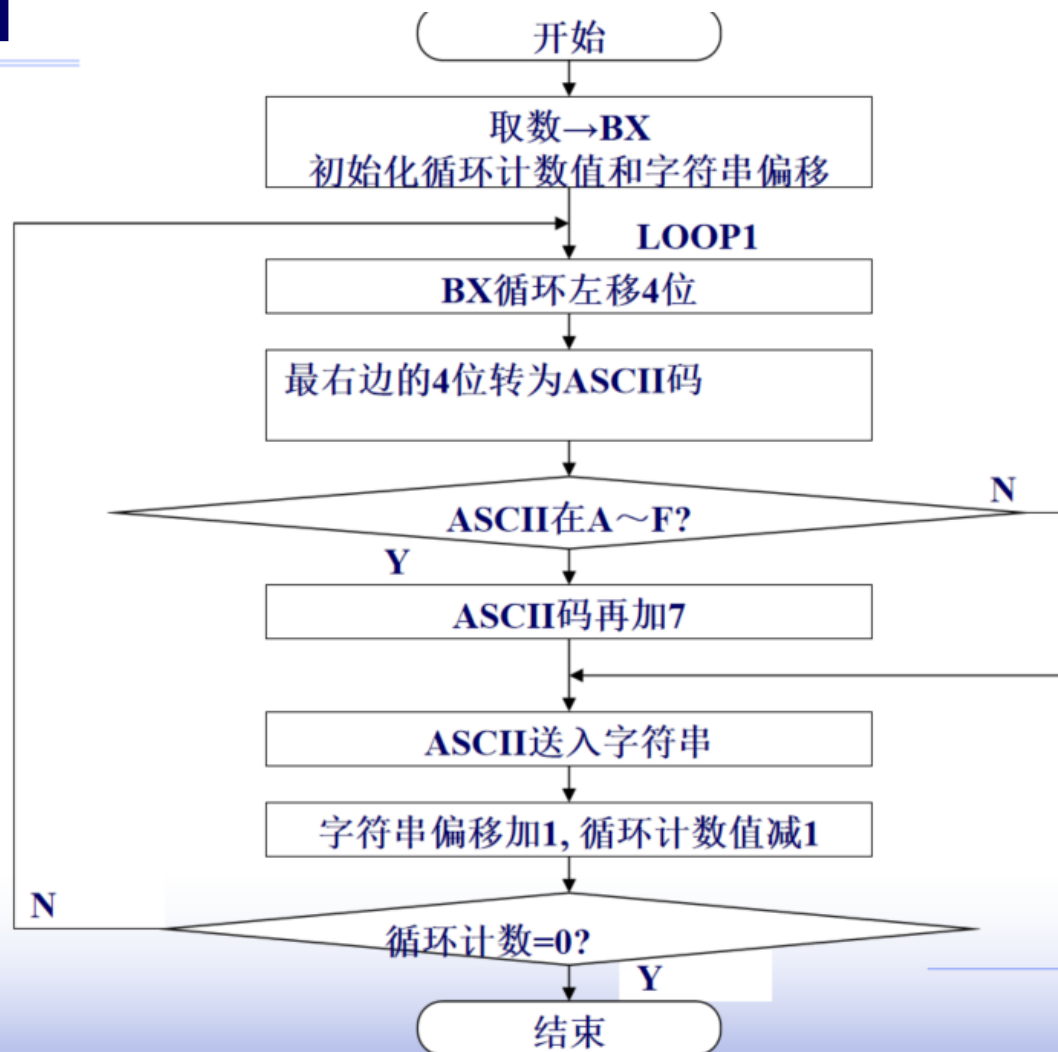
例：将BUF中定义的16位二进制数用十六进制数的形式显示在屏幕上。

分析：16位二进制数，每4位一组在屏幕上显示。用循环结构可以完成，每一个循环显示一个十六进制数，因此，循环次数已知，为4。循环中包含十六进制数与ASCII码字符的转换。



a) 循环次数已知

例：流程图





a) 循环次数已知

```
# AL, 用作ASCII字符转化
# BX, 用作循环移位
# CH, 用作循环计数
# EDI, 用作输出串变址
.section .data
buf1: .word 0x1234
buf2: .byte 0, 0, 0, 0, '\n'
.section .text
.global _start
_start:
    mov buf1, %bx
    mov $4, %ch
    mov $0, %edi
loop1:
    rol $4, %bx
    mov %bl, %al
    and $0xf, %al
```

```
    add $0x30, %al
    cmp $0x3a, %al
    jl out1
    add $7, %al
out1:
    mov %al, buf2(%edi)
    inc %edi
    dec %ch
    jnz loop1
    mov $4, %eax
    mov $1, %ebx
    mov $buf2, %ecx
    mov $5, %edx
    int $0x80
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```





b) 最大循环次数未知

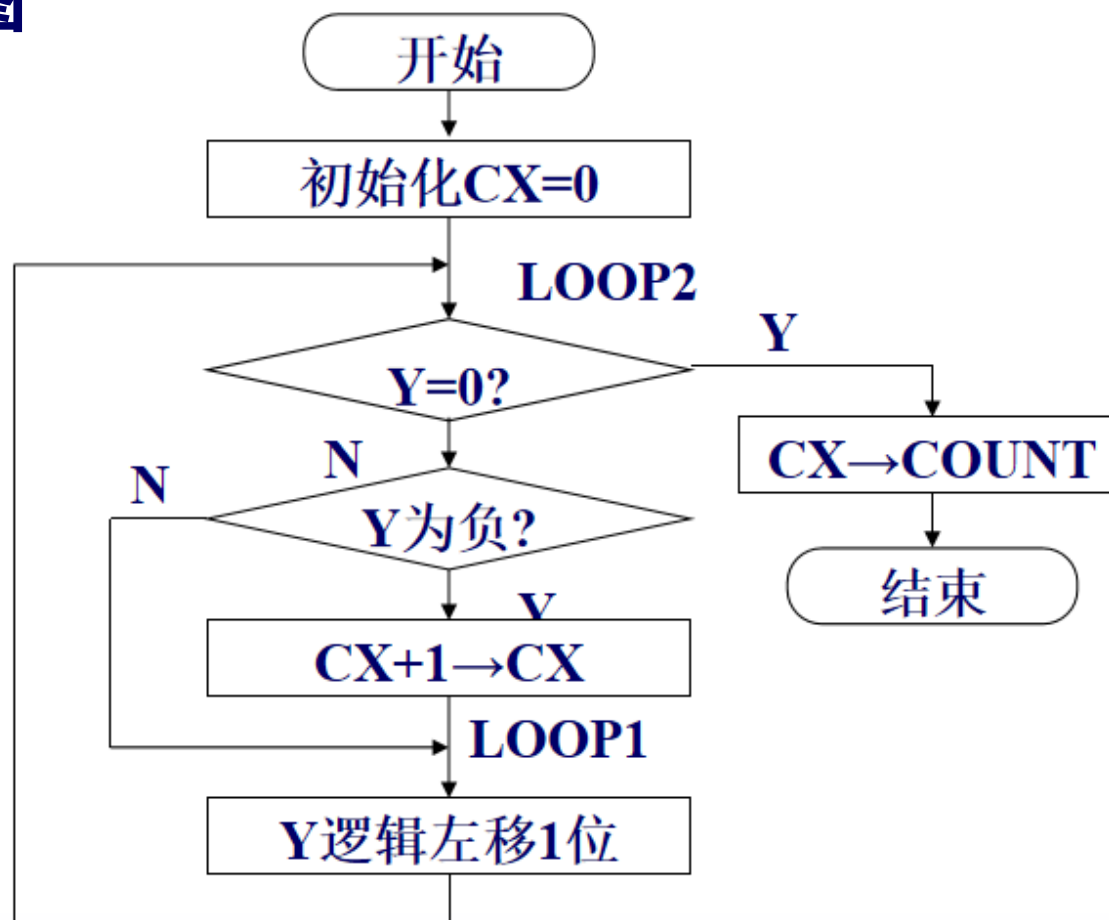
特点：用条件来控制循环。

例：将字变量Y中1的个数存入COUNT单元中。

分析：用移位的方法，将每位数移到最高位，如果Y=0就结束；如果为负数，则表示最高位为1。

b) 最大循环次数未知

例：流程图





b) 最大循环次数未知

```
# CX, 1的个数计数
# AX, 用作循环移位
.section .data
y: .word 0x55
count: .word 0
.section .text
.global _start
_start:
    mov $0, %cx
    mov y, %ax
loop2:
    test $0xffff, %ax
    jz exit
    jns loop1
    inc %cx
```

```
loop1:
    shl $1, %ax
    jmp loop2
exit:
    mov %cx, count

    mov $1, %eax
    mov $0, %ebx
    int $0x80
```




七、子程序

1.堆栈

(1)基本概念

a) 定义：主存中的一片数据存贮区

b) 用途：程序中经常用到子程序或处理中断，此时，主程序需要把子程序或中断程序将用到的寄存器内容保护起来，以便子程序或中断返回后，主程序能够从调用点或中断点处继续执行。





(1) 基本概念

c) 概念:

压入：数据进栈

弹出：数据出栈

栈底：堆栈的固定端

栈顶：栈指针指示的最后存入数据的单元

存取原则：先进后出。

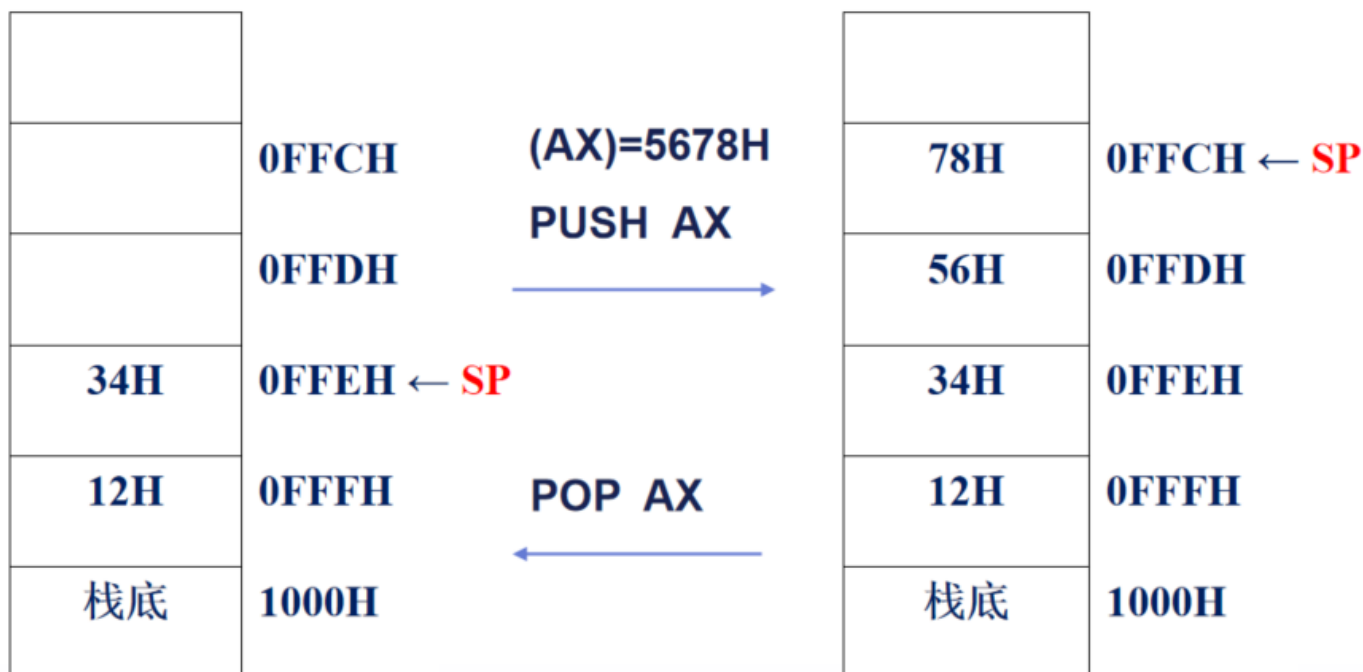


(1) 基本概念



华中科技大学

例：





(1) 基本概念

注意：

- **一端固定，一端活动**
- **存取“先进后出”**
- **操作单元大于字单元**
- **ESP栈顶指针**
- **ESP由高地址向低地址移动**



(2) 堆栈指令

a) 进栈指令

格式: **PUSH OPS**

OPS: 寄存器、段寄存器、存储器

功能: 将OPS的字数据压入堆栈。

进栈活动:

- **$SP - 2 \rightarrow SP$**
- **$OPS \rightarrow (\%SP)$**



(2) 堆栈指令

b) 出栈指令

POP OPD

OPD: 寄存器、段寄存器 (除CS)、存储器

功能: 将栈顶元素弹出到某一寄存器OPD。

出栈活动:

- **$(\%SP) \rightarrow OPD$**
- **$SP + 2 \rightarrow SP$**



(2) 堆栈指令

例：假定 AX=1234H, BX=5678H, CX=9ABCH

执行：

push %ax

push %bx

push %cx

pop

pop %ax

进栈：

.....	
BCH	← 0FFAH(SP)
9AH	0FFBH
78H	0FFCH
56H	0FFDH
34H	0FFEH
12H	0FFFH
栈底	← 1000H

出栈后：

.....	
.....	
.....	
.....	
.....	
34H	← 0FFAH(SP)
12H	0FFFH
栈底	← 1000H



(2) 堆栈指令

例：假定 AX=1234H, BX=5678H, CX=9ABCH

执行：

push %ax

push %bx

push %cx

pop %dx

pop %ax

结果：

DX=0x9ABC

AX=0x5678

BX=0x5678

CX=0x9ABC



(2) 堆栈指令

c) 8个寄存器进出栈指令

- 格式: PUSH A

功能: 将8个16位寄存器按AX、CX、DX、BX、SP、BP、SI、DI顺序入栈

- 格式: POP A

功能: 将8个16位寄存器按相反顺序出栈



c) 8个寄存器进出栈指令

- **格式: PUSHAD**

功能: 将8个32位寄存器按EAX、ECX、EDX、EBX、ESP、EBP、ESI、EDI顺序入栈

格式: POPAD

功能: 将8个32位寄存器按相反顺序出栈

2.子程序



华中科技大学

(1)基本概念

a) 概念:

一个程序中常常有类似的程序段，这些程序段的功能和结构相同，只是某些变量的赋值不同。可以把这些程序段写成具有独立功能的模块的形式，以便需要时调用它。这些模块称为子程序。



(1) 基本概念



华中科技大学

b) 子程序按实现者分类:

- 用户自定义子程序
- 系统子程序: 作为系统软件的一部分供用户调用。





(1) 基本概念

c) 子程序的优点:

- 程序模块化

将复杂程序划分为若干相对独立的模块，便于分工合作与管理

- 避免重复劳动

通过建立子程序库，提供标准子程序供多个程序共享使用，节省人力与时间

- 节省存储空间

子程序的复用避免了冗余的代码，减少了存储资源的占用

- 提高效率与质量

子程序设计使得程序简洁、清晰、易读，方便维护与修改，提高了程序设计的效率和质量。

- 实现数据交互

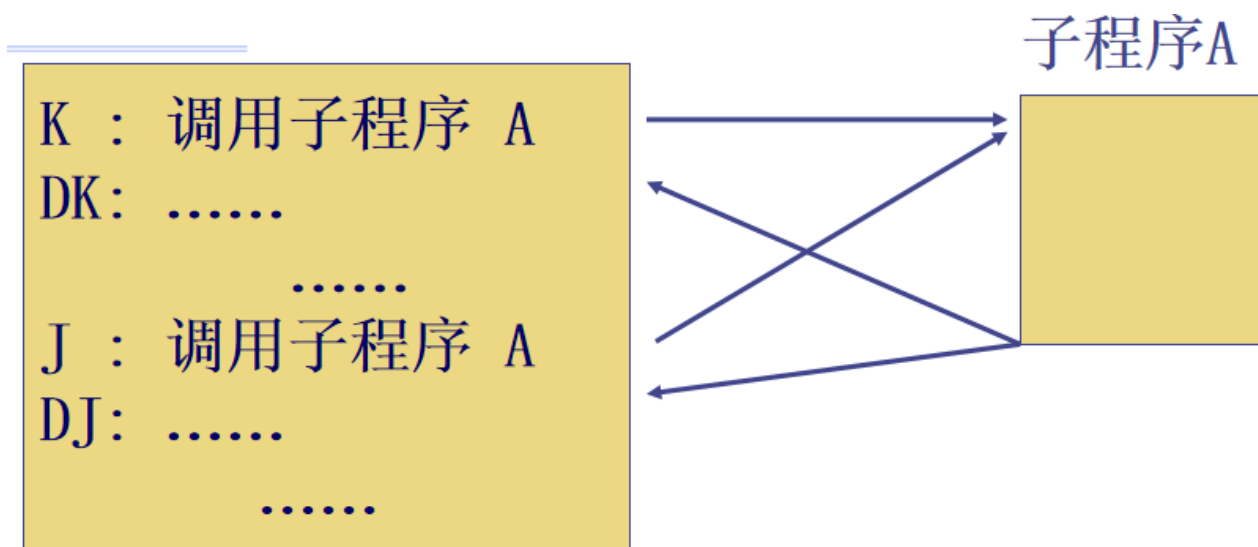
主程序通过传递参数给子程序，使得子程序能够加工数据并将结果返回给主程序。



(2)子程序与主程序的关系

调用与返回

主程序调用子程序，子程序执行结束后，又返回到主程序调用处的下一条指令。





(2)子程序与主程序的关系

子程序是独立的程序片段，可以用指令调用执行它。

这样的独立程序片段称为**子程序**。

调用子程序的程序称为**主程序**。在执行完子程序后，返回到调用它的主程序中继续执行。



(2)子程序与主程序的关系

- **调用的本质是什么？**

转移到子程序的第一行指令处，开始执行子程序的指令

- **返回的本质是什么？**

转移到主程序中，调用指令的下一行指令处，开始执行主程序的指令。

- **转移的本质是什么？**

修改EIP的值。将跳转目的地址，送入到EIP中。

- 如何实现“**调用-返回**”，使其按照预定的路线前进？

借助**堆栈**，保存返回点的地址；利用出栈指令，实现子程序的返回。





(3)子程序的定义

格式:

.type func_name, @function

func_name:

子程序体



(3)子程序的定义

例：定义一个名为sort的子程序

```
# call.s
```

```
.type sort, @function
```

```
sort:
```

```
...
```

```
ret
```





(4)子程序的调用和返回

a) 调用指令CALL

格式: **CALL** OPD

功能: 返回点地址压栈, 转移到目的地址执行。

按OPD的寻址方式, 分为:

- 直接调用
- 间接调用



a) 调用指令CALL

- 直接调用

OPD为标号，或地址值

例：

`call sort` ; sort为一个已定义的子程序

`call 0x8048933` ; 子程序sort的入口地址为
0x8048933，则此语句等价于 “`call sort`”



a) 调用指令CALL

- 间接调用

OPD为内存操作数。CALL指令以OPD的内容作为转移目的地址。

例：

movl \$sort, var_1 ; var_1是数据段中定义的一个变量。将sort的地址传送到var_1中

call * var_1 ; *表示是间接调用。OPD是内存操作数。由于var_1的内容为sort的地址，所以调用执行sort子程序

movl \$sort, var_1

mov \$var_1, %eax ; 将变量var_1的地址传送到EAX中

call * (%eax) ; *表示是间接调用。OPD是内存操作数。

由于(%eax)的内容是sort的地址，所以调用执行sort子程序





b) 返回指令ret

格式: RET

功能: 返回到调用指令的下一条指令处继续执行。

具体功能: 出栈到EIP

格式: RET n

功能: 出栈到EIP, 并继续出栈n个字节。

注意:

- **CALL和RET不影响标志位。**
- **对于RET n执行返回时, 清除堆栈中栈顶的n个字节。**

(5)现场保护和现场恢复



华中科技大学

调用子程序时，有可能破坏主程序中寄存器原来的内容，因此，必须保护、恢复现场。





(5) 现场保护和现场恢复

例：子程序f_1用到**EAX**、**EBX**寄存器的保护与恢复。

```
# protect.s
```

```
f_1:
```

```
    push    %eax
```

```
    push    %ebx
```

```
    ...    ; 子程序体
```

```
    pop     %ebx
```

```
    pop     %eax
```

```
    ret
```





(5) 现场保护和现场恢复

f_1:

```
push %eax
push %ebx
... ; 子程序体
pop %ebx
pop %eax
ret
```

注意:

- 在f_1的子程序中, 无论如何修改EAX、EBX, 返回主程序后, EAX、EBX都会保持原有的值不变。
- 按照堆栈“先进后出”的原则, 保护-恢复现场时, 寄存器出栈按寄存器进栈的逆序进行。
- 可以使用PUSHAD和POPAD, 保护-恢复所有寄存器。
- 若使用寄存器EAX存放子程序的返回值, 则不能对其进行保护和恢复。



(6) 参数传递

主程序为子程序提供入口参数，子程序返回结果给主程序。

a) 寄存器法

利用寄存器传送入口、出口参数，适合于参数少的情况。

该方法较简单，但参数个数受寄存器个数限制。

b) 约定单元法

入、出口参数在事先约定的存贮单元（全局变量）中。

适合于参数多的情况。





(6) 参数传递

c) 堆栈法

利用堆栈传递参数。

参数个数不受限制，并且适用于子程序嵌套调用的情况。

在C语言等高级语言中，子程序调用均主要采用堆栈法传递参数。

缺点是进出栈操作多，容易出错。





(6) 参数传递

例：将堆栈中的两元素相加

入口参数为栈顶和次栈顶的两个元素。

主程序：

```
# param.s
```

```
...
```

```
pushl x
```

```
pushl y
```

```
call f_sub
```

```
...
```

子程序：

```
f_sub:
```

```
mov %esp, %ebp
```

```
mov 4(%ebp), %ebx
```

```
mov 8(%ebp), %eax
```

```
add %ebx, %eax
```

```
ret
```



(6) 参数传递

主程序传参及调用后，堆栈的示意图如下：





子程序中堆栈的使用

- 调用和返回
- 现场保护和恢复
- 参数传递
- 局部变量



(7)子程序举例

例：从键盘输入一个十进制数，然后把该数以十六进制形式显示在屏幕上。

分析：

- **用子程序DECIBIN实现从键盘上取十进制数并转换为二进制。**
- **用子程序BINIHEX把二进制数以十六进制数的形式在屏上显示。**
- **参数传递通过寄存器EAX。**



(7) 子程序举例

```
.section .data
    in_buf: .byte 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0
    out_buf: .byte 0, 0, 0, 0, 0, 0,
0, 0, '\n'
    lf: .byte '\n'
.section .text
.global _start
_start:
    call decibin
    call binihex
    mov $1, %eax
    mov $0, %ebx
    int $0x80
.type decibin, @function
decibin:
    mov $3, %eax
    mov $0, %ebx
```

```
    mov $in_buf, %ecx
    mov $10, %edx
    int $0x80
    mov %eax, %ecx
    mov $0, %eax
    mov $0, %esi
    dec %ecx
next_1:
    movzxb in_buf(%esi), %ebx
    sub $'0', %ebx
    cmp $9, %ebx
    jg err_1
    cmp $0, %ebx
    jl err_1
    mov $10, %edx
    mul %edx
    add %ebx, %eax
    inc %esi..
```





(7)子程序举例

```
dec    %ecx
jnz    next_1
jmp    exit_1
err_1:
    mov    $0, %eax
exit_1:
    ret
.type   binihex, @function
binihex:
    mov    $8, %ecx
    mov    $0, %edi
next_2:
    rol    $4, %eax
    mov    %al, %dl
    and    $0xf, %dl
    add    $'0', %dl
    cmp    $0x3a, %dl
```

```
jl     label_2
add    $7, %dl
label_2:
    mov    %dl, out_buf(%edi)
    inc    %edi
    dec    %ecx
    jnz    next_2
    mov    $4, %eax
    mov    $1, %ebx
    mov    $out_buf, %ecx
    mov    $9, %edx
    int    $0x80
    ret
```